# From a Desired End-Effector Pose to Joint Torques: The Relative Cartesian Operational-Space Controller in OmniReset

**Tri Wahyu Guntara**
Tutorial notes on the UWLab OmniReset code base

**Abstract:** The reinforcement-learning policies in the OmniReset peg-insertion task do not command joint angles or joint torques. They command a small displacement of the end effector, and a controller converts that displacement into the torques applied at the six joints of a UR5e arm. This tutorial derives that controller from first principles. We construct the desired pose from the policy action, define the position and orientation errors, introduce the geometric Jacobian, write the task-space proportional–derivative law, and show by the principle of virtual work why the transposed Jacobian maps a task-space force to joint torques. We then discuss the gains, the two control rates, the actuator layer, and what the controller deliberately omits relative to the full operational-space formulation. Each equation is mapped to the corresponding lines of the implementation.

**Keywords:** Operational-space control, Jacobian transpose, Sim-to-real, Reinforcement learning

## 1 The Pipeline at a Glance

The policy in the OmniReset peg-insertion task [1] outputs a small displacement of the end effector, and a controller is responsible for producing the joint torques that realise that displacement. The controller is a *relative Cartesian operational-space controller*: relative, because the policy specifies a change of pose rather than an absolute pose; Cartesian, because the specification lives in the three-dimensional workspace rather than in joint space; and operational-space, because the control law is written in terms of a force on the end effector that is then mapped to the joints [2].

Figure 1 shows the whole chain. Two rates are involved. The policy runs once every $0.1\,\mathrm{s}$ and produces a new desired pose. Within that interval the physics engine advances twelve times, and at every one of those steps the controller recomputes the torques from the current state and the held desired pose.

## 2 Frames and Notation

Throughout, all poses, velocities, forces, and Jacobians are expressed in the robot's *base frame*, the frame of the UR5e's `base_link`. The end effector is the `wrist_3_link`, the flange to which the gripper is bolted; the gripper itself is not part of the controlled chain. The six arm joints are, in order, the shoulder pan, shoulder lift, elbow, and three wrist joints.

Let $q \in \mathbb{R}^6$ be the vector of arm joint angles and $\dot{q}$ its time derivative. The end-effector pose is the pair $x = (p, Q)$, where $p \in \mathbb{R}^3$ is the position and $Q$ is a unit quaternion, stored in $(x, y, z, w)$ order as elsewhere in the code base. A rotation can equally be written as a rotation vector $\theta\hat{n} \in \mathbb{R}^3$, the product of an angle and a unit axis; the controller moves between the two representations freely. $\Delta x \in \mathbb{R}^6$ denotes a small pose change, with its first three entries a translation and its last three a rotation vector. The subscript $\mathrm{des}$ marks a desired quantity, $K_p, K_d \in \mathbb{R}^{6\times6}$ are diagonal gain matrices, and $J(q) \in \mathbb{R}^{6\times6}$ is the geometric Jacobian.

Because the position and orientation both have three degrees of freedom, every task-space quantity in this controller is six-dimensional, and the arm has exactly six joints. The Jacobian is therefore square. This coincidence simplifies the exposition but is not essential: the control law below is valid for any number of joints.

## 3 From Action to Desired Pose

The policy emits an action $a \in \mathbb{R}^6$ whose entries are nominally of unit scale. The first operation multiplies it elementwise by a fixed scale vector $s$, which converts a dimensionless network output into metres and radians:

$$\Delta x = s \odot a, \qquad s_{\text{train}} = (0.02,\ 0.02,\ 0.02,\ 0.02,\ 0.02,\ 0.2). \tag{1}$$

During the first training stage a unit action therefore requests a 2 cm translation along any axis, a rotation of $0.02$ rad (about $1.1°$) about the base $x$ or $y$ axes, and a rotation of $0.2$ rad (about $11.5°$) about the base $z$ axis. The larger allowance about $z$ reflects the task: aligning a peg with a hole mostly requires turning the tool about its own axis.

The translational part of the desired pose is simply the current position plus the commanded displacement,

$$p_{\text{des}} = p + \Delta x_{1:3}. \tag{2}$$

The rotational part requires more care, because rotations do not add. The last three entries of $\Delta x$ form a rotation vector $\mathbf{r} = \Delta x_{4:6}$. Its magnitude is the rotation angle and its direction is the rotation axis,

$$\theta = \|\mathbf{r}\|, \qquad \hat{n} = \mathbf{r}/\theta \quad (\theta > 10^{-6}), \tag{3}$$

and the corresponding unit quaternion is obtained from the exponential map,

$$\Delta Q = \left(\hat{n} \sin \tfrac{\theta}{2},\ \cos \tfrac{\theta}{2}\right). \tag{4}$$

When $\theta$ is below the threshold, the axis is undefined and the code substitutes the identity rotation $\Delta Q = (0, 0, 0, 1)$. The desired orientation is then the composition

$$Q_{\text{des}} = \Delta Q \otimes Q, \tag{5}$$

where $\otimes$ is the quaternion (Hamilton) product. The order matters. Multiplying on the left applies $\Delta Q$ *after* $Q$ and therefore expresses the rotation in the base frame: a positive $\mathbf{r}$ about the base $z$ axis always turns the tool about the vertical, regardless of how the tool is currently oriented. Had the delta been multiplied on the right, it would have been interpreted in the tool's own frame.

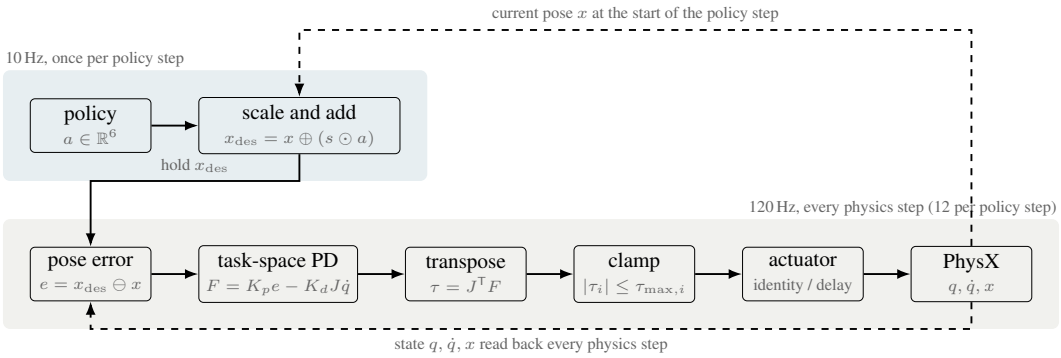



Figure 1: The control chain. The upper band runs once per policy step and only decides *where* the end effector should be. The lower band runs at the physics rate and decides *how hard to push* toward that place. Dashed arrows carry state read from the simulator.

**Why "relative" is a design decision.** Because $x_{\text{des}}$ is rebuilt from the *measured* pose at every policy step, the target can never run away from the robot. If contact with the hole stops the peg, the next policy step starts again from where the peg actually is, and the accumulated error is wiped out. This is what makes the scheme safe for contact-rich insertion: a stuck end effector never results in an ever-growing restoring force.

## 4 The Pose Error

At every physics step the controller reads the current pose $x = (p, Q)$ and compares it with the held target. The position error is the plain difference

$$e_p = p_{\text{des}} - p \in \mathbb{R}^3. \tag{6}$$

The orientation error is the rotation that takes the current orientation to the desired one. In quaternion form it is

$$Q_e = Q_{\text{des}} \otimes Q^{-1}, \tag{7}$$

which satisfies $Q_e \otimes Q = Q_{\text{des}}$, as required. This quaternion is converted back to a rotation vector by the logarithmic map, the inverse of the construction in Section 3:

$$e_o = \log(Q_e) = 2\ \operatorname{atan2}\big(\|\mathbf{v}\|, w\big)\ \frac{\mathbf{v}}{\|\mathbf{v}\|}, \qquad Q_e = (\mathbf{v}, w). \tag{8}$$

For small misalignments, $e_o$ is approximately twice the vector part of $Q_e$, and it behaves like an ordinary three-dimensional error signal: its direction is the axis about which the tool must turn and its magnitude is the remaining angle. The full six-dimensional error is the concatenation

$$e = \begin{bmatrix} e_p \\ e_o \end{bmatrix} \in \mathbb{R}^6. \tag{9}$$

Immediately after a policy step, before the arm has moved, the error equals the commanded delta, $e = \Delta x$. As the arm moves, the error shrinks toward zero.

## 5 The Jacobian

The geometric Jacobian links joint velocities to the linear and angular velocity of the end effector [3, 4],

$$\begin{bmatrix} v \\ \omega \end{bmatrix} = J(q)\,\dot{q}, \qquad J(q) \in \mathbb{R}^{6\times 6}. \tag{10}$$

Each column of $J$ is the end-effector twist produced by unit velocity at one joint while the others are held still. For a revolute joint $i$ with axis $\hat{z}_i$ through the point $o_i$, that column is

$$J_{:,i} = \begin{bmatrix} \hat{z}_i \times (p - o_i) \\ \hat{z}_i \end{bmatrix}, \tag{11}$$

where $p$ is the end-effector position. The cross product expresses the familiar fact that rotating about a distant axis sweeps the tool through a large arc, whereas rotating about a nearby axis barely moves it.

The controller does not query the physics engine for this matrix. It evaluates a closed-form expression derived from the UR5e's kinematic parameters, with the parameters replaced by values measured on the physical robot during calibration. This choice is deliberate. The real robot's controller uses exactly the same analytical Jacobian, so using it in simulation removes one source of discrepancy between the two. For the same reason the end-effector velocity entering the damping term is not read from the simulator but computed as $J\dot{q}$ from the measured joint velocities, which keeps the velocity and the Jacobian mutually consistent.

## 6 The Task-Space PD Law

With an error and a velocity in hand, the controller computes a *virtual wrench*, a force and a moment that one would like to apply to the end effector:

$$F = K_p\, e + K_d\big(\dot{x}_{\mathrm{des}} - J\dot{q}\big), \qquad \dot{x}_{\mathrm{des}} = 0. \tag{12}$$

The proportional term pulls the end effector toward the target in proportion to the remaining error, exactly as a spring would. The derivative term opposes the end-effector velocity, as a damper would. Because the target is held constant between policy steps, the desired velocity is zero and the damping term reduces to $-K_d J\dot{q}$. The analogy is therefore literal: the controller behaves as if six springs and six dampers, one per Cartesian direction, connected the tool to an invisible anchor at $x_{\mathrm{des}}$.

Both gain matrices are diagonal, so each direction is tuned independently. The stiffness matrix is specified directly; the damping is derived from it through a *damping ratio* $\zeta$,

$$K_d = 2\zeta\sqrt{K_p} \quad \text{elementwise.} \tag{13}$$

To see where this comes from, consider a single direction and suppose, for the moment, that the end effector behaved as a point of unit mass. Its motion would obey

$$\ddot{e} + K_d\dot{e} + K_p e = 0, \tag{14}$$

the equation of a damped harmonic oscillator with natural frequency $\omega_n = \sqrt{K_p}$. Writing $K_d = 2\zeta\omega_n$ gives the standard form $\ddot{e} + 2\zeta\omega_n\dot{e} + \omega_n^2 e = 0$, in which $\zeta$ alone decides the shape of the response: $\zeta < 1$ oscillates, $\zeta = 1$ returns to zero as fast as possible without overshooting, and $\zeta > 1$ returns more slowly and without overshoot. Section 9 shows what the two gain sets used in OmniReset look like under this model.

## 7 From Task-Space Force to Joint Torque

The wrench $F$ is a fiction: no actuator pushes on the wrist. What the robot can do is apply a torque $\tau_i$ at each joint. The question is which joint torques are equivalent to the desired wrench, and the answer is given by the principle of virtual work.

Suppose the joints move by an infinitesimal displacement $\delta q$. The end effector then moves by $\delta x = J\delta q$. The work done by the joint torques is $\tau^{\mathsf{T}}\delta q$; the work that the wrench would do through the same motion is $F^{\mathsf{T}}\delta x$. For the two to be equivalent, the work must agree for *every* possible $\delta q$:

$$\tau^{\mathsf{T}}\delta q = F^{\mathsf{T}} J\,\delta q \quad \forall\,\delta q \quad \Longrightarrow \quad \tau = J^{\mathsf{T}}F. \tag{15}$$

This is the central identity of operational-space control. The transpose of the matrix that maps joint velocities to end-effector velocities is the matrix that maps end-effector forces back to joint torques. No matrix inversion is required, and the formula remains valid even when $J$ is singular, which is one reason the transpose law is preferred over inverse-kinematics schemes for contact tasks.

The final operation is a safety clamp. Each component of $\tau$ is limited to the torque the corresponding joint can deliver,

$$\tau_i \leftarrow \mathrm{clip}\,\big(\tau_i, -\tau_{\max,i}, \tau_{\max,i}\big), \qquad \tau_{\max} = (150, 150, 150, 28, 28, 28)\ \mathrm{N}\cdot\mathrm{m}. \tag{16}$$

The three large joints of the UR5e are rated for $150\,\mathrm{N{\cdot}m}$ and the three wrist joints for $28\,\mathrm{N{\cdot}m}$. Clamping after the transpose, rather than clamping $F$, means that the limit is applied where it physically exists, at the joint, although it also means that a saturated joint changes the direction of the realised wrench.

## 8 A Planar Example

The six-dimensional case is hard to picture, but a two-link planar arm shows everything that matters. Let the links have lengths $\ell_1, \ell_2$ and joint angles $\theta_1, \theta_2$, with $\theta_2$ measured relative to the first link. The tip position is

$$p = \begin{bmatrix} \ell_1\cos\theta_1 + \ell_2\cos(\theta_1+\theta_2) \\ \ell_1\sin\theta_1 + \ell_2\sin(\theta_1+\theta_2) \end{bmatrix}, \tag{17}$$

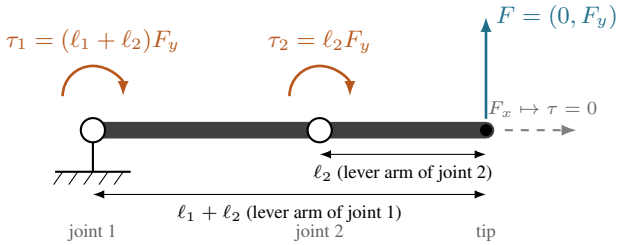



Figure 2: The transpose law on a two-link arm. The upward force at the tip is converted into joint torques equal to the force times each joint's lever arm. A force along the stretched arm maps to zero torque, because the arm cannot move that way.

Table 1: The two gain sets. The damping follows from the stiffness and the damping ratio by Eq. (13). Torque limits are 150 N·m on joints 1–3 and 28 N·m on joints 4–6 in both settings.

| | Training (stage 1) | | Evaluation / real robot | |
|---|---|---|---|---|
| Quantity | $x, y, z$ | $r_x, r_y, r_z$ | $x, y, z$ | $r_x, r_y, r_z$ |
| Stiffness $K_p$ | 200 | 3 | 1000 | 50 |
| Damping ratio $\zeta$ | 3.0 | 1.0 | 1.0 | 1.0 |
| Damping $K_d = 2\zeta\sqrt{K_p}$ | 84.9 | 3.46 | 63.2 | 14.1 |
| Action scale $s$ | 0.02 m | 0.02, 0.02, 0.2 rad | 0.01, 0.01, 0.002 m | 0.02, 0.02, 0.2 rad |

and differentiating with respect to the joint angles gives the Jacobian

$$J = \begin{bmatrix} -\ell_1 \sin\theta_1 - \ell_2 \sin(\theta_1 + \theta_2) & -\ell_2 \sin(\theta_1 + \theta_2) \\ \ell_1 \cos\theta_1 + \ell_2 \cos(\theta_1 + \theta_2) & \ell_2 \cos(\theta_1 + \theta_2) \end{bmatrix}. \tag{18}$$

Now take the configuration in Figure 2, with the arm stretched horizontally, $\theta_1 = \theta_2 = 0$. The Jacobian collapses to

$$J = \begin{bmatrix} 0 & 0 \\ \ell_1 + \ell_2 & \ell_2 \end{bmatrix}, \qquad J^{\mathsf{T}} = \begin{bmatrix} 0 & \ell_1 + \ell_2 \\ 0 & \ell_2 \end{bmatrix}. \tag{19}$$

Suppose the controller wants to push the tip upward with force $F = (0, F_y)$. Then

$$\tau = J^{\mathsf{T}} F = \begin{bmatrix} (\ell_1 + \ell_2) F_y \\ \ell_2 F_y \end{bmatrix}. \tag{20}$$

Each joint receives the force times its lever arm to the tip, which is exactly the torque a static force at the tip would exert on that joint. The shoulder, being farther away, must work harder than the elbow. If instead the controller wants to push along the arm, $F = (F_x, 0)$, the result is $\tau = 0$: the stretched arm cannot move in that direction, so no torque can help, and the transpose law quietly produces none. An inverse-Jacobian controller would have divided by zero at the same configuration.

## 9 Gains, Damping Ratios, and the Step Response

OmniReset uses two gain sets, listed in Table 1. The first, used throughout the initial training stage, is deliberately soft and heavily overdamped. The second, used for evaluation and for the real robot, is five times stiffer in translation and critically damped. During fine-tuning a curriculum interpolates from the first to the second.

The unit-mass model of Section 6 lets us compare the two settings. For the training gains in translation, the characteristic equation $s^2 + 84.9\, s + 200 = 0$ has roots near $-2.4$ and $-82$. The fast root dies out almost immediately; the slow root gives a time constant of about 0.4 s. For the evaluation gains, $s^2 + 63.2\, s + 1000 = 0$ has a double root at $-31.6$, a time constant of about 0.03 s.

Figure 3 integrates the model numerically for a step of one action unit. The vertical rules mark the 0.1 s policy period. With the training gains, the end effector has covered only about 19% of

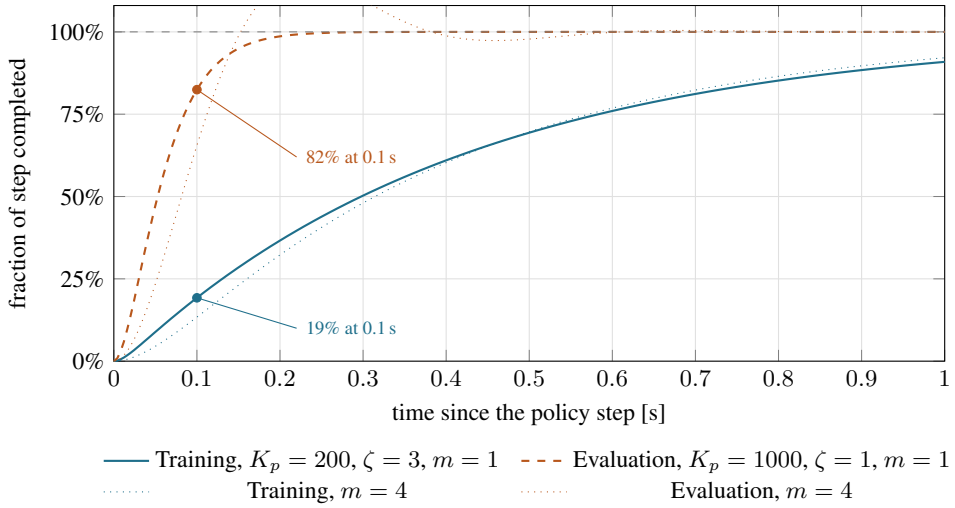



Figure 3: Fraction of a commanded translation step completed as a function of time, for the two gain sets, under an effective end-effector mass $m$ of 1 kg (solid, dashed) and 4 kg (dotted). The grid lines at multiples of 0.1 s are policy steps.

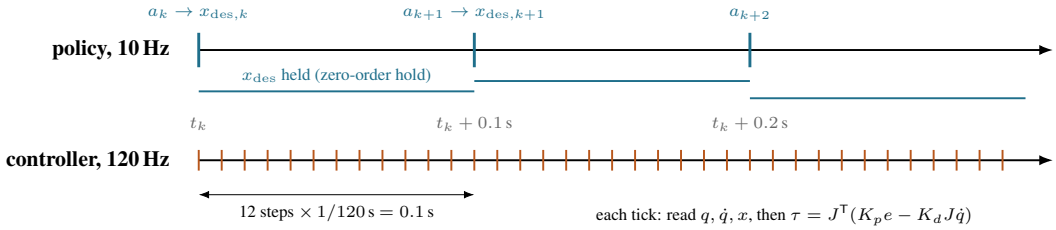



Figure 4: The two clocks. A policy action fixes the target for the next twelve physics steps; the torque is recomputed at each of them from the latest measurement. The held target is drawn at a different height after each tick to emphasise that it is rebuilt from the measured pose, not accumulated.

the commanded 2 cm by the time the next action arrives; the commanded delta functions less as a position target than as a velocity command, which is a sensible thing for an exploring policy to control. With the evaluation gains, the arm completes about 82% of the (smaller) step within one policy period, so the policy commands positions in a more literal sense. Under a heavier effective mass both curves slow down; the real arm's apparent mass varies with its configuration, which is the price of omitting the inertia matrix discussed in Section 12.

The rotational gains deserve a remark. A stiffness of 3 N·m/rad with the training set is tiny: an orientation error of a full radian produces only 3 N·m of moment at the wrist. This is intentional during exploration, since a compliant wrist survives collisions with the hole gracefully, and the fine-tuning curriculum raises the value to 50 N·m/rad once the policy is competent.

## 10 Timing: Two Control Rates

The simulation runs at a time step of $1/120$ s and the environment applies a *decimation* of 12, so the policy acts once every $12/120 = 0.1$ s. Figure 4 lays the two clocks side by side. At each policy tick the desired pose is recomputed from the measured pose and then held constant, a zero-order hold. In between, the controller runs twelve times, each time with a fresh measurement of $q$, $\dot{q}$ and $x$ but the same $x_{\mathrm{des}}$.

Running the feedback loop at the physics rate rather than at the policy rate is what makes the soft gains workable. A spring-damper evaluated only ten times a second would be unstable for the

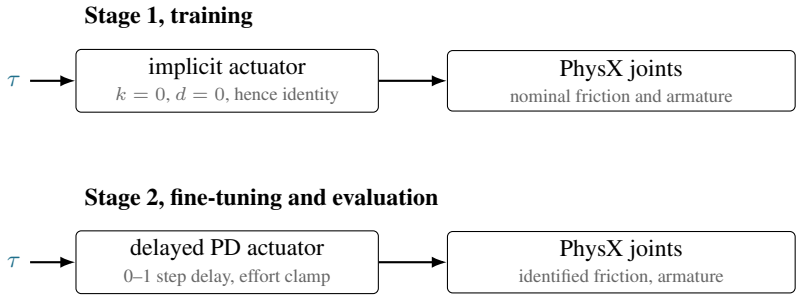



Figure 5: The actuator layer in the two training stages. In neither stage does the actuator add a joint-level PD term. The second stage inserts a one-step motor delay and loads the joint friction and rotor inertia identified on the physical arm, so that the same controller faces a more realistic plant.

damping values in Table 1, because $K_d\,\Delta t = 84.9 \times 0.1 \approx 8.5$ far exceeds the stability margin of a discrete damper. At $\Delta t = 1/120$ s the same product is about $0.7$, which is comfortably stable.

## 11 The Actuator Layer

In Isaac Lab [5] the quantity produced by an action term is a *target*, and an actuator model decides what the simulator finally receives. For a position-controlled robot the actuator would contain its own joint-level PD loop with its own stiffness and damping. The OmniReset arm sets both of those to zero. The effort target coming out of the operational-space controller is therefore passed to the physics engine unchanged, and the only PD loop in the system is the task-space one described above. Figure 5 shows the layer in the two training stages.

The second training stage replaces the implicit actuator by a *delayed* one. Its joint-level gains are still zero, so it still does not add a PD term; what it adds is a delay of zero or one physics step between the torque being computed and its being applied, together with an explicit effort clamp. In the same stage a randomisation event writes the joint friction and armature values identified on the real UR5e [6] into the simulator, scaled by a factor drawn from a range that a curriculum widens as the policy's success rate permits. The controller itself does not know about any of this; it only notices that the plant has become harder to move.

## 12 What the Controller Leaves Out

Readers familiar with Khatib's operational-space formulation [2] will have noticed an omission. The complete law reads

$$\tau = J^{\mathsf{T}}\Lambda(q)\big(K_p e - K_d J\dot{q}\big) + \mu(q,\dot{q}) + g(q), \tag{21}$$

where $\Lambda(q) = \big(JM^{-1}J^{\mathsf{T}}\big)^{-1}$ is the task-space inertia matrix built from the joint-space mass matrix $M(q)$, and $\mu$ and $g$ compensate Coriolis, centrifugal, and gravitational effects. Pre-multiplying by $\Lambda$ makes the closed loop behave exactly like the unit-mass oscillator of Section 6 in every direction and every configuration. The OmniReset controller drops $\Lambda$, $\mu$, and $g$ entirely, so the law is

$$\tau = J^{\mathsf{T}}\big(K_p e - K_d J\dot{q}\big) \tag{22}$$

and nothing more. Three consequences follow.

1. **The apparent stiffness depends on the configuration.** Without $\Lambda$, the same gain produces a fast response in directions where the arm is light and a slow one where it is heavy. This is why Figure 3 shows two effective masses: no single curve describes the real arm.
2. **Gravity must come from elsewhere.** In simulation the robot links are spawned with gravity disabled, so there is nothing to compensate. On the physical UR5e the manufacturer's firmware compensates gravity below the level at which torques are commanded, so again the controller need not.

3. **Simulation and reality use the same equations.** This is the reason for the simplification. The controller running on the real robot is this same transpose PD law with the same analytical Jacobian and the same gains. A more sophisticated simulated controller would have trained the policy against a plant that behaves differently from the one it meets on deployment.

## 13 Mapping the Mathematics to the Code

The controller is implemented in the class `RelCartesianOSCAction` in the file `source/uwlab_tasks/uwlab_tasks/manager_based/manipulation/omnireset/mdp/actions/task_space_actions.py`. Its two methods correspond to the two bands of Figure 1. The first runs once per policy step and implements Section 3:

```
def process_actions(self, actions):
    scaled = actions * self._scale                          # dx = s * a
    ee_pos_b, ee_quat_b = self._get_ee_pose_root_frame()    # x = (p, Q) in the base frame
    self._ee_pos_des[:] = ee_pos_b + scaled[:, :3]          # p_des = p + dx[0:3]

    delta_rot = scaled[:, 3:6]                              # rotation vector r
    angle = torch.norm(delta_rot, dim=-1, keepdim=True)     # theta = |r|
    axis = delta_rot / angle      # guarded for theta ~ 0   # n = r / theta
    delta_quat = cat([axis * sin(angle / 2), cos(angle / 2)])   # dQ = (n sin, cos)
    self._ee_quat_des[:] = quat_mul(delta_quat, ee_quat_b)      # Q_des = dQ * Q
```

The second runs at every physics step and implements Sections 4 through 7 in order:

```
def apply_actions(self):
    ee_pos_b, ee_quat_b = self._get_ee_pose_root_frame()
    joint_pos = self._asset.data.joint_pos[:, self._joint_ids]        # q
    joint_vel = self._asset.data.joint_vel[:, self._joint_ids]        # dq

    jacobian = compute_jacobian_analytical(joint_pos)                 # J(q), calibrated
    ee_vel = bmm(jacobian, joint_vel)                                 # xdot = J dq

    pos_error = self._ee_pos_des - ee_pos_b                           # e_p
    quat_error = quat_mul(self._ee_quat_des, quat_inv(ee_quat_b))     # Q_e = Q_des * Q^-1
    pose_error = cat([pos_error, axis_angle_from_quat(quat_error)])   # e = [e_p; log Q_e]

    task_force = self._kp * pose_error + self._kd * (-ee_vel)         # F = Kp e - Kd xdot
    joint_torques = bmm(jacobian.transpose(-1, -2), task_force)       # tau = J^T F
    joint_torques = clamp(joint_torques, -self._torque_max, self._torque_max)
    self._asset.set_joint_effort_target(joint_torques, joint_ids=self._joint_ids)
```

Two implementation details complete the picture. In the constructor the damping is derived from the stiffness exactly as in Eq. (13), and both gains are stored per environment so that a randomisation event can scale them independently in each of the thousands of parallel copies of the scene. On reset, the desired pose is set equal to the current pose, so that a freshly reset environment starts with zero error and the arm does not lurch toward a stale target. The gain values themselves live in `config/ur5e_robotiq_2f85/actions.py`, in three configurations: one for training, one for evaluation and the real robot, and an unscaled one for the system-identification scripts, where the action is interpreted directly in metres and radians.

## 14 Limitations

These notes describe the controller as implemented on the `feature/forge-omnireset` branch of the UWLab repository in October 2026 and may drift from later revisions. The step-response analysis in Section 9 uses a lumped unit-mass model that ignores the configuration dependence of the task-space inertia, the torque clamp, joint friction, and the motor delay, so its time constants are indicative rather than predictive. The orientation error is treated as a small-angle signal; for large rotations the rotation-vector error is not a linear function of the quaternion. Finally, the tutorial does not cover how the real-robot controller implements the same law on the UR5e firmware.

## References


[1] UW Lab. Emergent dexterity via diverse resets and large-scale reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2603.15789.

[2] O. Khatib. A unified approach for motion and force control of robot manipulators: The operational space formulation. *IEEE Journal on Robotics and Automation*, 3(1):43–53, 1987.

[3] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo. *Robotics: Modelling, Planning and Control*. Springer, 2009.

[4] K. M. Lynch and F. C. Park. *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017.

[5] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, D. Hoeller, J. L. Yuan, R. Singh, Y. Guo, H. Mazhar, A. Mandlekar, B. Babich, G. State, M. Hutter, and A. Garg. Orbit: A unified simulation framework for interactive robot learning environments. *IEEE Robotics and Automation Letters*, 8(6): 3740–3747, 2023.

[6] F. Bjelonic et al. PACE: Physics-aware calibration for sim-to-real transfer. *arXiv preprint arXiv:2509.06342*, 2025.